Guide 3 · Viva Defence

Questions you will be asked, strong honest answers, and how to deliver them

AgriAgent Final Year Project · 11 October 2026

> **How to use this.** Do not memorise the answers word for word; examiners notice. Understand the **point** of each answer, then say it in your own words. Practise out loud with a teammate asking questions in random order. Answers marked **[update]** must be refreshed once real results exist.

1 How to answer any question

1. **Answer first, then explain.** "Yes, because…" not a five-minute run-up.
2. **Point at evidence.** "The test `test_spray_export_safety.py` checks exactly that." "Here is the real run."
3. **Separate real from mock** every time you mention a number.
4. **Own the weaknesses.** "That's a known limitation; here is why, and here is what we did about it."
5. **If you don't know:** "I'm not sure. I would check it by…" Never invent.

2 The pitch

2.1 30 seconds

"Farmers spray herbicide over whole fields, wasting chemical and harming soil. AgriAgent takes a photo of a field and produces a map of only the weeds, so only those spots would be sprayed. It uses open-vocabulary vision models, Grounding DINO and SAM 2, that need no training on our data. A set of deterministic safety rules decides what goes on the map, so an AI mistake cannot directly cause spraying, and herbicide advice comes from the official ICAR bulletin rather than from a language model."

2.2 2 minutes (add)

"The pipeline has five stages: detect, segment, gate, escalate and output. [Walk the diagram.] Our first real run on a cotton field taught us the key lesson: a generic prompt cannot separate weed from crop. The detector boxed the whole field and the crop was marked for spraying. So our evaluation centres on one safety metric, how much of the crop ends up on the spray map, and our improvement work targets that. [update: our result is …]"

3 Motivation and novelty

Q: What problem are you solving, and why does it matter?

Blanket herbicide spraying wastes chemical, costs farmers money and pollutes soil and water. Spot spraying reduces this, but commercial systems need large labelled datasets per crop. We explore whether general-purpose models can produce safe selective spray maps without that training.

Q: What is novel here? Grounded-SAM already exists.

Correct: combining Grounding DINO and SAM is known (Grounded-SAM). Our contribution is the system around it for a safety-critical agricultural use: a deterministic, tested safety gate between the models and the spray output; fail-closed handling of crop and unknown labels; escalation to a reasoning step for uncertain cases; herbicide advice grounded in Indian statutory documents with banned-chemical and dose checks; and an honest evaluation that measures crop damage, not only mask quality.

Q: Why not just train YOLO or a U-Net on a weed dataset?

That is the standard approach and would likely be more accurate on that one crop. Our question is different: how far can you get **without** per-crop training, which matters for crops and regions with no labelled data, such

as Indian cotton. A trained model is the natural baseline to compare against, and we name it as future work. [update if a baseline is added]

Q: Who would use this?
In its current form, researchers and agronomists reviewing spray maps. It is a research prototype, not a product: it does not control a sprayer and its coordinates are not calibrated.

4 Vision

Q: How does Grounding DINO work?
It encodes the image (Swin Transformer) and the text prompt (BERT), fuses them with cross-attention so regions matching words are strengthened, and decodes boxes, each scored against each prompt phrase. The box's label is the phrase it matched best.

Q: How does SAM 2 work, and why do you need it?
It is promptable segmentation: given an image and a box or points, it returns the object's exact mask and a predicted IoU score. Boxes include soil and neighbouring plants; spraying a box would hit the crop. The mask lets us spray only the weed's pixels.

Q: What is the refinement loop?
If SAM 2's predicted IoU is below 0.85, we compute a point where the mask likely missed foliage and ask SAM 2 again with that point, at most twice. The idea comes from MedSAM-Agent. Honest note: predicted IoU is the model's self-estimate, and we have not yet shown with real data that refinement improves true IoU; our mock tests cannot show it because the mock adds a fixed improvement.

Q: What prompt do you use?
Default `"weed . grass . pigweed ."`. Prompt design matters a lot: our real run showed generic prompts match all vegetation. Prompt and the gate's label rules must be designed together. [update: the prompt strategy that worked best]

Q: Why the tiny model versions?
They fit an 8 GB laptop GPU and are fast, which matches the edge-device motivation. Larger versions are an obvious accuracy experiment.

5 Safety

Q: How do you guarantee the crop is never sprayed?
We do not claim a guarantee, and no vision system can give one. What we guarantee in software, and test, is: only masks whose label passes a fail-closed policy (no crop words, must be a known weed word) reach the map; anything unknown is excluded; maps with more than 85% infestation are suppressed entirely; and the export uses only these approved masks, never raw detector boxes. The remaining risk is the model mislabelling crop as weed, which is exactly what our crop-on-map metric measures.

Q: Why deterministic rules instead of letting the AI decide?
Rules give the same answer every time, can be read line by line, and can be tested exhaustively. A neural network can be confidently wrong in ways you cannot list in advance. For a physical action involving chemicals, the final authority should be something you can verify. Models advise; rules decide.

Q: Walk me through the safety gate.
[Recite the six rules in order from Guide 2, Section 5.2.] Safety first, then quality, then uncertainty; the first matching rule wins.

Q: Why 85%?
If almost the whole image is "weed", the detection has probably failed (like our real run) or spot spraying no longer makes sense and broadcast is appropriate. 85% is a chosen threshold, not derived from data; it should be tuned with an agronomist.

Q: You found a safety bug. Tell me about it.
The workflow filtered masks correctly, but the app's GeoJSON exporter rebuilt spray zones from raw boxes and only checked if the label contained "weed", so a label like "cropweed" was exported. We fixed it so the exporter reads only the approved zones from the workflow, and added regression tests for each case. It is a good example of why the export path must share one source of truth.

Q: Are there remaining weaknesses in the gate?
Yes, and we documented them. The final gate call passes confidence and IoU as constants, so only the infestation rule really applies at that point; and escalation happens after the map is built without withdrawing it. [update: fixed in commit …] Naming these is part of our evaluation.

Q: What is Clef Flash, and does it run?
A 9-billion-parameter decision model from Cloudflare, built on Qwen3.5-9B. It answers typed questions with a probability per option instead of writing text, which suits triage: "spray, escalate or skip?". We call it on Cloudflare Workers AI (free allowance, accepts images), with a 4-bit local copy through llama.cpp as an offline backup. It only advises: deterministic checks run first, and if Clef fails or is unsure, the mask is escalated, never sprayed. [update: say whether it ran in your results.]

6 Agronomy and RAG

Q: What is System 2?
The slower path for uncertain cases: a vision-language model names the weed and growth stage as validated JSON, then a retrieval step looks up the matching recommendation in the ICAR-DWR bulletin.

Q: Why RAG? Why not ask the LLM which herbicide to use?
LLMs can hallucinate chemicals, including banned ones, and wrong doses. Retrieval grounds the answer in an official document. In our code the chemical and dose come from the retrieved row, then pass hard checks: banned list, dose ceiling. If nothing relevant is retrieved, we return "no knowledge found" and suggest nothing.

Q: How do you know the banned list and ceilings are right?
They were transcribed from the bulletin's tables into code. [update: verified by hand against the PDF on date …; or: verification is pending, and that is a stated limitation.]

Q: What embedding model and vector store?
`BAAI/bge-small-en-v1.5` embeddings (384 dimensions) in ChromaDB. Tests use an offline hash embedder so they run without downloads.

Q: Which LLM do you use?
Groq-hosted `qwen/qwen3.8-27b` in the cloud and `Qwen3.5-4B` locally through llama.cpp, plus Clef Flash for triage. [update: say which one actually ran in your results.]

7 Evaluation and results

Q: What are your results?
[update with the real table.] Until then, the honest answer: "Our first real run showed failure on weed-versus-crop separation; we have a 20-image annotated evaluation set and are measuring detection quality, mask IoU, crop pixels on the spray map, and herbicide savings."

Q: How do you measure success?
Against human annotations: precision and recall of weed detection, IoU and Dice of masks, the percentage of crop pixels included in the spray map (our safety metric), and herbicide savings versus spraying the whole image. Every number is labelled with model, device and whether it was a real run.

Q: Your earlier documents claimed much better results.
Some early documents, partly AI-generated, presented mock outputs too confidently. We audited everything, removed mock results, and now label every claim as real, mock, partial, planned or blocked. We'd rather report a modest real result than an impressive fake one.

Q: What is mock mode and why does it exist?
Stand-ins that return fixed outputs so the pipeline can be tested on any laptop quickly and repeatably. It tests the plumbing and safety logic, not accuracy. We never report mock numbers as results.

Q: How many tests do you have, and what do they prove?
175 unit and 35 end-to-end tests, run by CI on Python 3.10 and 3.11 with linting and type checks. They prove the logic (for example that crop labels can never reach the export), not that the models are accurate.

8 Engineering

Q: Why LangGraph?
The pipeline is a graph with a loop (refinement) and a conditional branch (escalation). LangGraph expresses that directly. We also kept a plain-Python runner with the same order, so tests and the app do not depend on the library.

Q: Explain `GraphState`.
One typed dictionary holding everything the pipeline knows. Each stage reads some keys and returns updates. That makes each stage independently testable.

Q: How did you make the code testable without a GPU?
Every model sits behind a wrapper with a mock mode, and providers share an interface, so tests can swap in stand-ins.

9 The uncomfortable questions

Q: Did you write this code yourself? Was it AI-generated?
Be truthful. A strong version: "We used AI coding assistants heavily, as many developers now do. We learned that this creates risks: inflated documentation and code we hadn't reviewed. So we audited the whole codebase, removed what was wrong, and each of us can explain our part. Ask me about any file." Then **be able to back it up**. That is what Guide 2 is for.

Q: What did you personally contribute?
Prepare a specific answer per person, with files and decisions: "I designed/wrote X, found and fixed Y, ran experiment Z."

Q: Your real run failed. Isn't the project a failure?
No. It is the project's central finding: open-vocabulary detection with generic prompts does not separate weed from crop. Identifying that, measuring it properly and showing what reduces it is a legitimate research result. [update: and here is what reduced it]

Q: Could this be deployed on a real sprayer?
Not yet. It would need calibrated GPS, camera-to-nozzle timing, real-time speed measurements, hardware interlocks, agronomist validation and supervised field trials. We list these as future work.

Q: What would you do differently?
Get one real end-to-end result in week one before building features; freeze model choices early; keep one source-of-truth document; never let generated documents claim results without evidence.

10 Track-specific questions

10.1 Track 1 · Vision and safety

- Trace one image through `run()`.
- Why does `"cotton weed"` get excluded? Is that right?
- What happens when the GPU is missing? (Silent mock fallback: known issue, and how you'd fix it.)

10.2 Track 2 · Agronomy

- Walk through `create_chemical_prescription` step by step.
- What does the VLM output, and how is it validated?
- What happens if Groq is down or the key is missing?

10.3 Track 3 · Data and evaluation

- Describe the evaluation set: images, annotations, crop vs weed classes.
- How do you compute crop pixels on the spray map from box annotations?
- Why report confidence intervals with 20 images?

10.4 Track 4 · UI and integration

- How does the app show whether it is in mock or live mode?
- Where does the GeoJSON come from, and why not from boxes?
- What does CI check, layer by layer?

11 Self-check

Answer each out loud in under one minute, without notes:

1. The 30-second pitch.
2. What is novel?
3. Why deterministic rules?
4. Why RAG for herbicides?
5. The safety bug you fixed.
6. What your real run showed and why.
7. What mock mode is and is not.
8. Your personal contribution.